

Big Data Platform for Financial Fraud, Money-Laundering, and Market Anomaly Detection using Hadoop and Spark

First Review

Group-7

B Dinesh Reddy– CB.AI.U4AID24107

Jagath Varun V- CB.AI.U4AID24121

K Charan Teja- CB.AI.U4AID24124

K Praveen – CB.AI.U4AID24166

23AID302 – Big Data Analytics
Department of Artificial Intelligence and Data Science

Introduction to the Problem Domain

- Our project is in the area of **Big Data Analytics applied to Finance** – specifically fraud detection, money laundering detection, and unusual behaviour in stock trading.
- Banks and stock markets create huge amounts of data every second: card transactions, bank transfers, and stock price ticks.
- Why this matters:
 - Financial fraud causes billions of dollars in losses every year.
 - Sudden unusual trading activity can cause market crashes.
- Older, rule-based systems check transactions one at a time and cannot handle this much data at this speed, or catch new kinds of fraud they were not specifically told to look for.
- Big data tools like Hadoop and Spark are built to process this volume of data quickly and can help catch fraud and unusual trading patterns faster.

Problem Statement

Problem Statement

Current fraud detection, money-laundering detection, and market anomaly detection systems are usually built as separate projects, each using one small, static dataset and one method. Most of them do not combine a trained classifier with information from the transaction network (who is connected to whom), and most do not check their results against real, known cases to see how accurate they actually are. Our project builds one combined system that works on two different kinds of financial data at once, processes the data as a continuous stream instead of one-time batch, and checks its results properly instead of just assuming they are correct.

Objectives

- Develop a scalable Hadoop–Spark framework for real-time financial analytics using IBM AML and stock market datasets.
- Construct transaction networks using Spark GraphFrames and extract graph-based features for enhanced fraud analysis.
- Design a Graph-based Dynamic Risk Score (GDRS) to evaluate the financial risk of each transaction before classification.
- Detect fraudulent transactions, money laundering activities, and stock market anomalies using Spark MLlib machine learning models.
- Improve transparency using Explainable AI (SHAP/LIME) to provide interpretable fraud predictions for financial decision-making.

Literature Review

Paper	Method	Limitation
Finding Money Launderers using Heterogeneous Graph Neural Networks (2025)	Uses Heterogeneous Graph Neural Networks (HGNN) to model financial transaction networks for Anti-Money Laundering (AML) detection.	Focuses only on AML detection. Does not support stock market anomaly detection, dynamic risk scoring, explainable AI, or a unified Hadoop–Spark framework.
Big Data-Driven Fraud Detection using Machine Learning and Real-Time Stream Processing (2025)	Uses Apache Spark, Kafka, and Machine Learning models for real-time fraud detection on streaming financial data.	Detects only financial fraud. Does not perform graph feature engineering, financial risk fusion, or integrated fraud, AML, and market anomaly detection.

Research Gap

- Existing works focus on individual financial problems such as Fraud Detection or AML.
- They lack graph-based feature engineering, Graph-based Dynamic Risk Score (GDRS), Financial Risk Fusion, Explainable AI, and an integrated Hadoop–Spark framework.
- Our work proposes a unified financial analytics framework that integrates all these components for comprehensive financial risk analysis.**

5Vs of Big Data

- **Volume:** Processes large-scale IBM AML transaction records and minute-level stock market datasets using Hadoop HDFS.
- **Velocity:** Spark Structured Streaming enables near real-time processing of continuous financial transactions and stock market updates.
- **Variety:** Integrates banking transactions, stock market time-series data, and graph-based transaction networks using Spark GraphFrames.
- **Veracity:** Improves data reliability through preprocessing, graph feature engineering, Graph-based Dynamic Risk Score (GDRS), and model validation using Precision, Recall, F1-score, and ROC-AUC.
- **Value:** Generates actionable financial insights through Fraud Detection, AML Detection, Market Anomaly Detection, Graph-based Dynamic Risk Score (GDRS), Financial Risk Index, and Explainable AI, enabling faster and more informed financial decision-making.

Proposed System (1/3)

Overview

The proposed framework integrates Fraud Detection, Anti-Money Laundering (AML), and Stock Market Anomaly Detection using a unified Hadoop–Spark architecture.

- Store IBM AML and Stock Market datasets in Hadoop HDFS.
- Process financial data using Spark Structured Streaming.
- Perform data preprocessing including cleaning, normalization and feature extraction.
- Construct transaction networks using Spark GraphFrames.
- Generate graph-based features for intelligent financial analysis.

Proposed System (2/3)

Novel Framework Components

① Graph Feature Engineering

- Extract Degree Centrality
- Compute PageRank
- Detect suspicious transaction communities

② Graph-based Dynamic Risk Score (GDRS)

- Calculate financial risk using
 - Transaction Amount
 - Transaction Frequency
 - Graph Features
 - Historical Fraud Behaviour
 - Transaction Time

③ Train Random Forest / Gradient Boosted Trees using both original features and GDRS.

Proposed System (3/3)

Financial Intelligence Framework

- ① Detect Fraudulent Transactions.
- ② Detect Anti-Money Laundering (AML) activities.
- ③ Detect Stock Market Anomalies.
- ④ **Financial Risk Fusion Engine**
 - Combine Fraud Score
 - Combine AML Score
 - Combine Market Risk Score
 - Generate a unified Financial Risk Index
- ⑤ **Explainable AI (SHAP/LIME)**
 - Explain why each transaction is classified as suspicious.
 - Improve transparency and support financial decision making.

Final Outputs

- Fraud Detection
- AML Detection
- Market Anomaly Detection

Implementation Details

Hadoop Cluster

- 1 NameNode + 3 DataNodes
- HDFS 3.2.1
- Replication factor: 3
- Docker + WSL2

HDFS Dataset

- Raw data: 22.2 GB
- 11 files
- Block size: 256 MB
- 97 HDFS blocks
- 291 block replicas

Processing

- Apache Spark 3.5.0
- Spark Standalone
- 2 workers
- 8 cores
- Scala 2.12.18

Data Flow

Raw CSV/JSON → HDFS → Spark + Scala →
Parquet → HDFS

Results & Analysis

Full-Scale Processing

Dataset	Rows	Flagged	Time (s)
HI-Medium	31.90M	35,230	77.1
LI-Medium	31.25M	16,041	48.2
HI-Large	179.70M	225,546	257.8
Market SPY	2.07M	726,969	12.6
Total	244.92M	-	429.3

Output

- CSV and JSON processed
- Converted to Snappy Parquet
- Typed and partitioned
- Stored back in HDFS

Validation

- Scala and PySpark compared
- HI-Medium: 31,898,238 rows
- Laundering rows: 35,230
- Results matched exactly

Challenges / Limitations

- **Large-scale data:** Processing 244.9 million financial and market records requires high computational and storage resources.
- **Data preprocessing:** Raw CSV and JSON data require extensive cleaning, schema definition, type conversion, and transformation.
- **Duplicate columns:** The CSV contains two columns named “Account”, requiring an explicit schema for correct processing.
- **Large value range:** Transactions across multiple currencies have widely varying values, requiring USD normalization and log transformation.
- **Resource management:** Spark jobs required memory and resource configuration to execute successfully.

10. Big Data Tools Used

Tool	Purpose
Hadoop HDFS	Distributed storage
Hadoop YARN	Resource management
Apache Spark	Distributed processing
Scala	Data preprocessing
PySpark	Future analytics and ML
Apache Parquet	Columnar data storage
Docker Compose	Cluster deployment

Impact

- Processes 244.9 million rows using distributed computing
- 97 input blocks enable parallel processing
- Parquet improves analytical data access
- Partitioning reduces unnecessary data scans
- Replication provides fault tolerance

Impact Comparison: Big Data vs Conventional

Aspect	Conventional	Our Platform
Data size	Limited by RAM	Distributed across HDFS
Processing	Single process	Parallel Spark tasks
Storage	Raw CSV	Columnar Parquet
Data access	Full-row processing	Column-based access
Partitioning	Full scan	Date/year pruning
Fault tolerance	Limited	3-way replication
Scaling	Larger machine	Add DataNodes

Overall Impact

Distributed storage + parallel processing + efficient columnar storage enables large-scale analytics.

References

- ① Ananthu S., N. Sethumadhavan, and H. Narayanan A. G., "Credit Card Fraud Detection using Apache Spark Analysis," in *Proc. 2021 5th Int. Conf. Trends in Electronics and Informatics (ICOEI)*, Tirunelveli, India, 2021. DOI: 10.1109/ICOEI51242.2021.9453011
- ② X. Li, X. Cao, X. Qiu, J. Zhao, and J. Zheng, "Intelligent Anti-Money Laundering Solution Based upon Novel Community Detection in Massive Transaction Networks on Spark," in *Proc. 2017 5th Int. Conf. Advanced Cloud and Big Data (CBD)*, Shanghai, China, 2017. DOI: 10.1109/CBD.2017.38
- ③ M. Ahmed, N. Choudhury, and S. Uddin, "Anomaly Detection on Big Data in Financial Markets," in *Proc. 2017 IEEE/ACM Int. Conf. Advances in Social Networks Analysis and Mining (ASONAM)*, Sydney, Australia, 2017.
- ④ IBM, "IBM Transactions for Anti Money Laundering (AML)," Kaggle. Available: <https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>
- ⑤ "Intraday Stock Data (1 min) – S&P 500 – 2008–21," Kaggle. Available: <https://www.kaggle.com/datasets/gratefuldata/intraday-stock-data-1-min-sp-500-200821>

Thank You